FAULT TOLERANT P2P DISTRIBUTED STORAGE SYSTEM

Implementation Manual

Student: [Student Name]
Roll: [Roll Number]
Institution: [Institution]
Department: [Department]
Guide: [Guide Name]
Academic year: [Academic Year]

Document index

1. Project overview

2. Objectives and scope

3. System architecture

4. Directory and module guide

5. Database and metadata

6. Upload workflow

7. Retrieval workflow

8. Replication placement

9. Failure and repair

10. Security and integrity

11. Configuration

12. Installation

13. Run commands

14. HTTP and TCP protocol

15. Tests and verification

16. Performance evaluation

17. Troubleshooting

18. Limitations and future work

19. Conclusion

20. References

Evidence note: prior audit reports 11 unit tests passing. Benchmarks and end-to-end failure recovery were not executed.

1. Project overview

Educational local distributed storage prototype. Flask coordinates encrypted chunk uploads/downloads, SQLite tracks metadata, and three TCP nodes hold data. N1/N2 start ACTIVE; N3 starts STANDBY.

2. Objectives and scope

Store encrypted chunks, retain two replicas, reconstruct downloads with integrity checks, and detect stale nodes and attempt standby repair. Source implements these mechanisms; package evidence does not include end-to-end measurement.

3. System architecture

CLI uses HTTP with coordinator/app.py. Storage logic is coordinator/storage.py, metadata coordinator/db.py, nodes node/node_process.py. TCP node requests and heartbeat messages use length-prefixed JSON. HTTP port 5000, heartbeat 6100, nodes 6001-6003 on loopback.

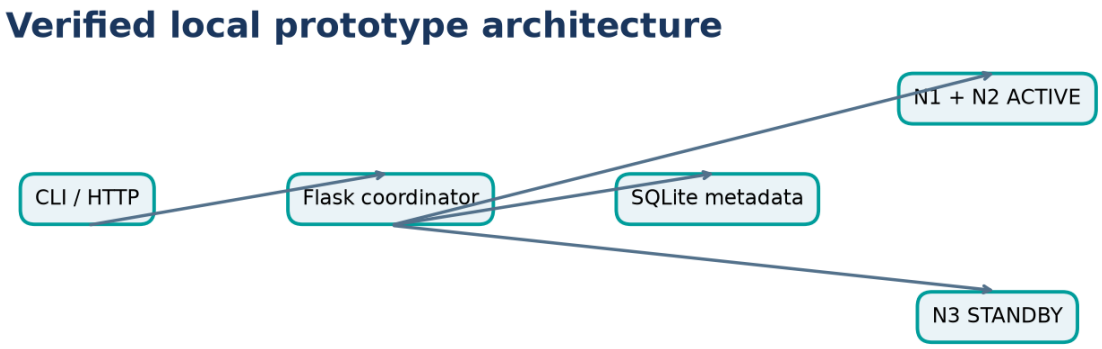


Figure: System architecture

4. Directory and module guide

client/cli_client.py: HTTP client. coordinator/app.py: API and startup. storage.py: upload/download pipeline. chunking.py: 1 MiB chunks. crypto.py: AES-GCM/SHA-256. placement.py: deterministic pair. node_client.py: TCP STORE/FETCH. node/node_process.py: storage server/heartbeats. heartbeat_monitor.py: health detection. recovery.py: standby repair. db.py: schema. merkle.py: Merkle root. audit.py: hash-chain. tests/: component tests. experiments/: benchmark harnesses.

5. Database and metadata

SQLite tables: files(file_id PK, filename, size_bytes, chunk_count, merkle_root, created_at); chunks(chunk_id PK, file_id FK, chunk_index, chunk_hash); nodes(node_id PK, host, port, status, last_heartbeat); replicas(chunk_id,node_id composite PK, role); audit_log(entry_index PK,event,prev_hash,entry_hash,created_at). Foreign keys enabled. P2P_DB_PATH overrides default coordinator/metadata.db.

6. Upload workflow

POST /files multipart field file. Requires two ACTIVE nodes; split plaintext into 1 MiB chunks, AES-256-GCM encrypt each with random 12-byte nonce/16-byte tag, hash encrypted data, calculate Merkle root, place on two nodes via STORE, and persist metadata after all acknowledgements. Failed partial writes may leave orphan node files.

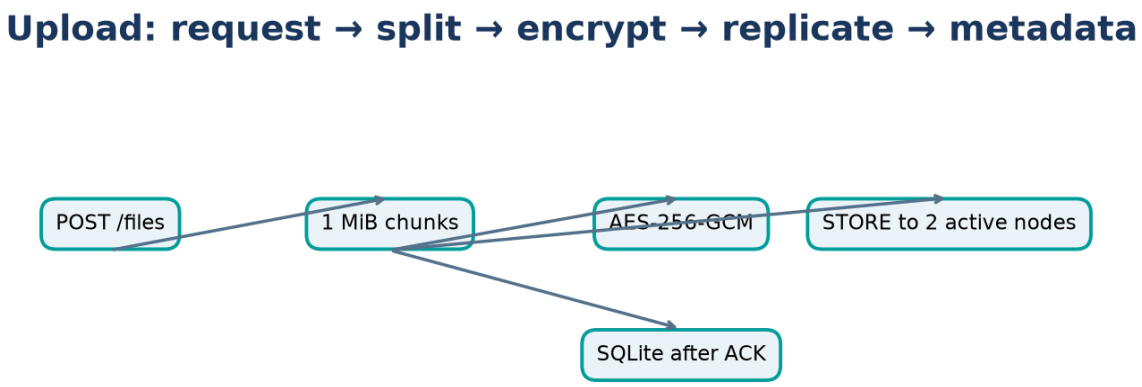


Figure: Upload workflow

7. Retrieval workflow

GET /files/<id> loads ordered chunks and replicas. Try nodes not marked DOWN; FETCH, compare encrypted SHA-256, decrypt/authenticate AES-GCM, verify Merkle root, reassemble, and verify original byte count. Missing usable replica causes error.

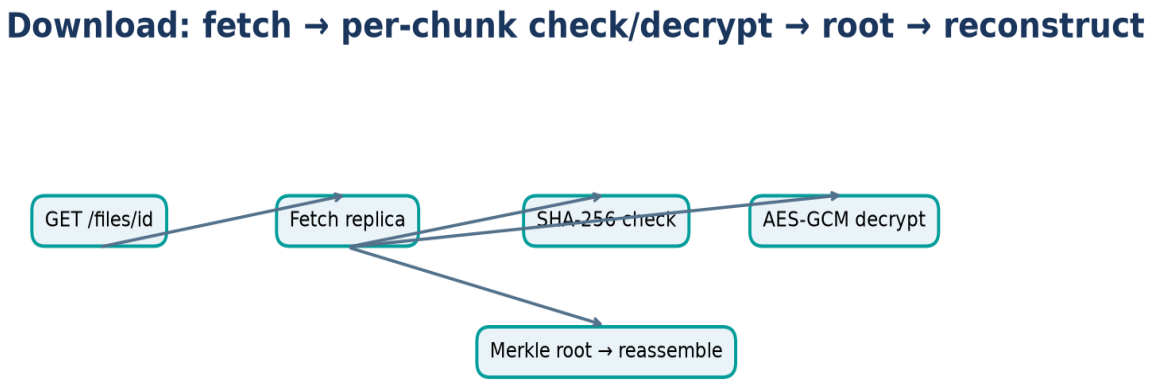


Figure: Retrieval workflow

8. Replication placement

SHA-256(chunk ID) first eight bytes big-endian modulo sorted ACTIVE node count chooses primary; next circular position is secondary. Initial two active nodes both receive each chunk. Fewer than two ACTIVE nodes prevents upload.

9. Failure and repair

Heartbeats every 1 s; ACTIVE node older than 4 s after first heartbeat is marked DOWN. Recovery selects a STANDBY, fetches affected data from a surviving non-DOWN replica, stores on standby, remaps only after ACK, then promotes standby. No standby means no repair. Restarted DOWN nodes are not automatically reintegrated.

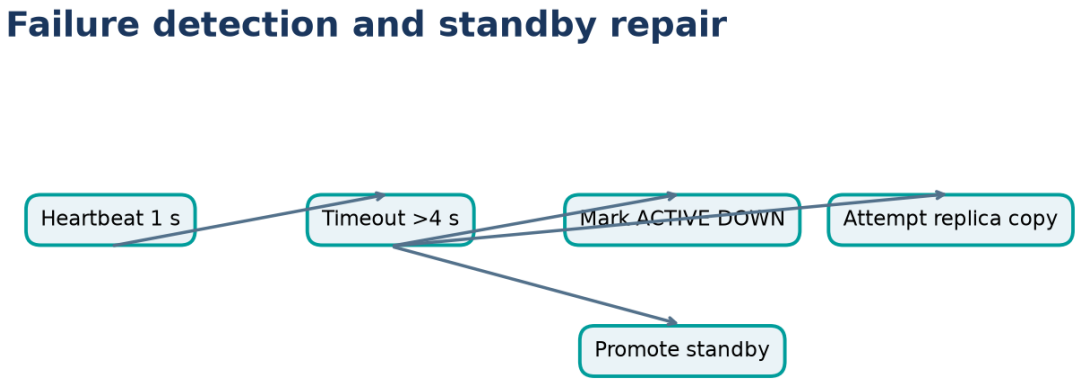


Figure: Failure and repair

10. Security and integrity

AES-256-GCM blob is nonce(12)||ciphertext||tag(16). Static demo key in coordinator/config.py; no external key management. HTTP/TCP lack authentication/TLS. SHA-256 checks encrypted chunks; Merkle combines digest strings. GCM authenticates ciphertext. Audit entries link hashes but are not externally immutable.

11. Configuration

config.py: heartbeat 1 s, timeout 4 s, port 6100, chunk size 1 MiB, factor 2, static key, node ports 6001-6003, DB path coordinator/metadata.db or P2P_DB_PATH. HTTP port 5000 in app.py. node CLI requires --node-id and --port; coordinator host/port optional defaults loopback/6100.

12. Installation

Python 3.10+ per README. From repo root: python -m venv .venv; PowerShell .venv\Scripts\Activate.ps1; python -m pip install -r requirements.txt. Dependencies are Flask, PyCryptodome, pytest, Matplotlib, Requests.

13. Run commands

Four terminals: python coordinator/app.py; python node/node_process.py --node-id N1 --port 6001; repeat N2/6002 and N3/6003. Wait for heartbeats. CLI: python client/cli_client.py status; upload path; download FILE_ID output. Ctrl+C to stop.

14. HTTP and TCP protocol

HTTP routes: POST /files, GET /files/<file_id>, GET /status. TCP messages use 4-byte big-endian length and UTF-8 JSON. STORE sends chunk id and base64 bytes; FETCH sends id. One request per connection. Heartbeat is a separate TCP listener.

15. Tests and verification

Prior audit reported `pytest -q`: 11 passed; not rerun during package generation. Reproduce from repository root with `pytest -q` after installing requirements. Existing tests cover chunking, crypto, placement/schema, Merkle/audit, heartbeat detection. They do not establish benchmark performance or end-to-end recovery. Benchmark commands are `python experiments/bench_upload_retrieval.py` and `python experiments/bench_failure_recovery.py`; use an isolated clone, ensure fixed ports are free, and preserve existing results before execution.

16. Performance evaluation

No numeric performance results. Existing upload benchmark defaults to 4096/65536/1048576 bytes, 3 trials, validates equality. Failure benchmark uses 16384 bytes and measures DOWN and N3 ACTIVE transitions over 3 trials. Neither was run because scripts use fixed ports and write into source tree.

17. Troubleshooting

Dependencies: activate venv/install requirements. Port bind: inspect ports before use and do not stop unknown processes. Node status: check coordinator and node host/port/heartbeats. Upload 503: requires two ACTIVE nodes. Download failure: check replica records and nodes. Hash mismatch skips replica; repair is not automatic for corruption. Keep P2P_DB_PATH consistent.

18. Limitations and future work

Single coordinator/SQLite, loopback defaults, no discovery/consensus/auth/TLS/key management/garbage collection/multi-coordinator HA/partition policy. One-node repair depends on surviving source and standby. No automatic reintegration. Future: secure keys, authenticated transport, resilient metadata, multi-host tests.

19. Conclusion

Source implements encrypted replicated chunk storage and basic heartbeat-triggered standby repair. Existing unit tests were reported passing. No performance or end-to-end recovery outcome is claimed here.

20. References

Flask docs: https://flask.palletsprojects.com/ ; SQLite: https://www.sqlite.org/docs.html ; Python sockets: https://docs.python.org/3/library/socket.html ; PyCryptodome AES: https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html ; NIST FIPS 180-4: https://doi.org/10.6028/NIST.FIPS.180-4 ; pytest: https://docs.pytest.org/ ; Matplotlib: https://matplotlib.org/stable/